

Manifold Dynamics

Technical Documentation

Version 0.1.0-alpha

Physics-Based Terrain Analysis Platform

Document Status: Alpha Release

This documentation covers the core architecture and implementation of the Manifold Dynamics platform as of November 2025. The system is under active development and validation.

November 19, 2025

Contents

1	Introduction	4
1.1	Overview	4
1.1.1	Core Capabilities	4
1.1.2	Design Philosophy	4
1.2	System Requirements	5
1.2.1	Software Dependencies	5
1.2.2	Hardware Requirements	5
2	Architecture	6
2.1	System Overview	6
2.2	Core Modules	6
2.2.1	manifold.core.io	6
2.2.2	manifold.core.ingest	6
2.2.3	manifold.physics.kesm	7
3	MFT File Format Specification	8
3.1	Format Overview	8
3.2	Binary Structure	8
3.2.1	File Header	8
3.2.2	Data Blocks	8
3.3	Channel Layout	9
3.4	File Size Calculation	9
3.5	Python API	9
4	Mathematical Foundations	11
4.1	MFCO: Manifold-Fit Comfort Optimizer	11
4.1.1	Motivation	11
4.1.2	Structure Tensor Computation	11
4.1.3	Eigenvalue Decomposition	12
4.1.4	Derived Metrics	12
4.1.5	Implementation Details	12
4.2	KESM: Katabatic-Entropic Stability Model	13
4.2.1	Physical Basis	13
4.2.2	Full Advection-Diffusion Model	13
4.2.3	KESM-Lite: Laplacian Proxy	14
4.2.4	Classification	14
4.2.5	Implementation	14

4.3	Auxiliary Terrain Metrics	15
4.3.1	Topographic Position Index (TPI)	15
4.3.2	Vector Ruggedness Measure (VRM)	16
5	Data Processing Pipeline	17
5.1	Ingestion Workflow	17
5.2	Destriping Algorithm	17
5.2.1	Method: Median-Based Offset Alignment	17
6	Usage Examples	19
6.1	Basic Processing	19
6.1.1	Convert GeoTIFF to MFT	19
6.1.2	Load and Inspect MFT File	19
6.1.3	Run KESM Analysis	19
6.2	Visualization	20
6.2.1	Interactive Viewer (Napari)	20
6.2.2	Static Plots (Matplotlib)	20
6.2.3	KESM Visualization	21
6.3	Batch Processing	22
7	Performance Optimization	23
7.1	Memory Management	23
7.1.1	Memory-Mapped Access	23
7.1.2	Tiled Processing	23
7.2	GPU Acceleration	24
7.2.1	CuPy Integration	24
7.2.2	C++ Extension	24
7.3	Benchmark Results	25
8	Validation and Testing	26
8.1	Unit Tests	26
8.1.1	Running Tests	26
8.1.2	Test Structure	26
8.2	Validation Protocol	27
8.2.1	Field Deployment Checklist	27
8.2.2	Validation Metrics	28
9	Troubleshooting	30
9.1	Common Issues	30
9.1.1	Integer Elevation Data	30
9.1.2	Tile Stitching Artifacts	30
9.1.3	Memory Errors	30
9.1.4	KESM False Positives	30
9.1.5	Napari Rendering Issues	31
9.2	Getting Help	31
9.2.1	Reporting Issues	31
9.2.2	Community Resources	31

10 Future Development	32
10.1 Roadmap	32
10.1.1 Version 0.2.0 (Q1 2026)	32
10.1.2 Version 0.3.0 (Q2 2026)	32
10.1.3 Version 1.0.0 (Q4 2026)	32
10.2 Research Directions	33
10.2.1 Time-Dependent Phenomena	33
10.2.2 Multi-Physics Coupling	33
10.2.3 Machine Learning Integration	33
11 Appendices	34
11.1 Appendix A: File Format Specification	34
11.1.1 MFT Version History	34
11.1.2 Future Extensions	34
11.2 Appendix B: Mathematical Notation	35
11.3 Appendix C: Hardware Benchmarks	35
11.3.1 CPU Performance	35
11.3.2 GPU Performance	35
11.4 Appendix D: License Information	35
11.4.1 Core Library	35
11.4.2 Physics Engines	36
11.4.3 Third-Party Dependencies	36
11.5 Appendix E: Citation	36

1 Introduction

1.1 Overview

Manifold Dynamics is an open-source platform for physics-based terrain analysis that moves beyond traditional geometric GIS operations to incorporate differential geometry, thermodynamics, and computational fluid dynamics. Unlike conventional GIS tools that perform simple geometric calculations (slope, aspect), Manifold Dynamics models the physical phenomena that occur on terrain surfaces.

1.1.1 Core Capabilities

The platform currently implements three primary analytical engines:

- **MFCO (Manifold-Fit Comfort Optimizer):** Eigenvalue-based surface roughness analysis using Principal Component Analysis of the terrain structure tensor
- **KESM (Katabatic-Entropic Stability Model):** Thermodynamic prediction of cold air drainage and frost hollow formation using discrete Laplacian analysis
- **MFT Format:** Custom multi-channel tensor format optimized for physics-aware terrain processing

1.1.2 Design Philosophy

Traditional GIS treats terrain as a static geometric surface. Manifold Dynamics treats terrain as a *dynamic physical system* where:

- Surface geometry influences fluid flow (air, water)
- Material properties affect traversability
- Topographic position determines micro-climate phenomena
- Multi-scale analysis captures phenomena from centimeters to kilometers

1.2 System Requirements

1.2.1 Software Dependencies

Package	Version	Purpose
Python	3.12+	Core runtime environment
NumPy	1.26.0+	Array operations and linear algebra
SciPy	1.11.0+	Scientific computing, Laplacian solver
Rasterio	1.3.0+	Geospatial raster I/O
GDAL	3.8.0+	Coordinate transformation, format support
Napari	0.4.18+	GPU-accelerated visualization
Matplotlib	3.8.0+	Static plotting and exports
PyQt6	6.6.0+	GUI framework (optional)

Table 1.1: Core Dependencies

1.2.2 Hardware Requirements

Minimum Configuration:

- CPU: 4 cores, 2.5 GHz
- RAM: 8 GB
- Storage: 10 GB free space
- GPU: Not required (CPU fallback available)

Recommended Configuration:

- CPU: 8+ cores, 3.0+ GHz
- RAM: 32 GB
- Storage: 100 GB SSD
- GPU: NVIDIA with CUDA support (for large datasets)

2 Architecture

2.1 System Overview

The Manifold Dynamics architecture consists of three primary layers:

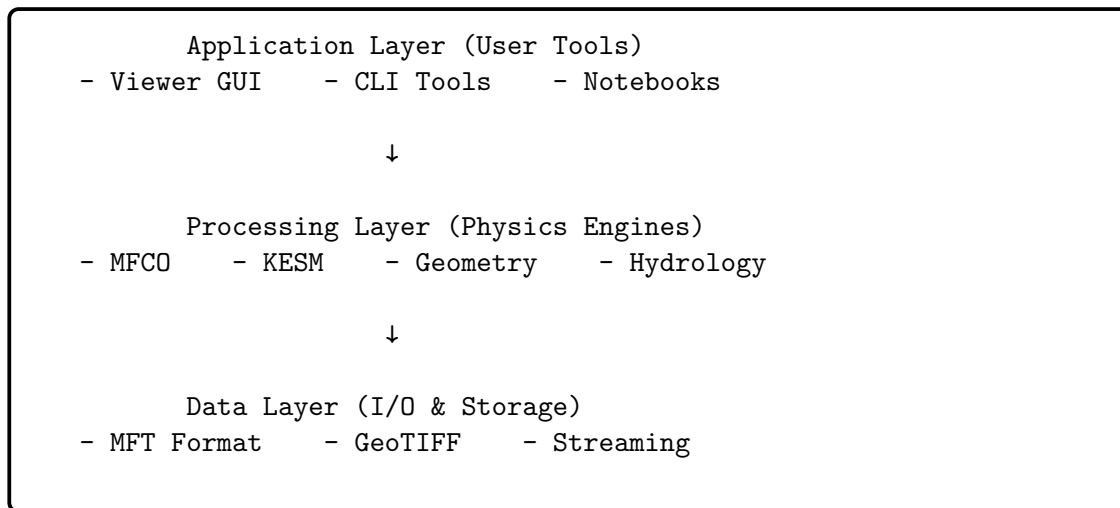


Figure 2.1: Manifold Dynamics Architecture

2.2 Core Modules

2.2.1 manifold.core.io

Handles all data input/output operations, including the proprietary MFT format and standard geospatial formats.

Key Classes:

- `ManifoldTensor`: Core data structure representing multi-channel terrain data

2.2.2 manifold.core.ingest

Converts external geospatial data (GeoTIFF, ADF, etc.) into the MFT format, computing all derived channels.

Key Functions:

- `ingest_geotiff()`: Main ingestion pipeline

- `destripe_terrain()`: Artifact removal
- `compute_eigen_roughness()`: MFCO implementation
- `calculate_tpi()`: Topographic Position Index
- `calculate_vrm()`: Vector Ruggedness Measure

2.2.3 `manifold.physics.kesm`

Implements the Katabatic-Entropic Stability Model for thermodynamic terrain analysis.

Key Classes:

- `KESM_Lite`: Discrete Laplacian-based cold air drainage solver

3 MFT File Format Specification

3.1 Format Overview

The Manifold Tensor Format (.mft) is a custom binary format designed for efficient storage and access of multi-channel terrain data. Unlike GeoTIFF which stores each derived product as a separate file, MFT packages all related channels into a single file with predictable memory layout.

3.2 Binary Structure

3.2.1 File Header

The MFT file begins with a 96-byte fixed header:

Offset	Type	Field	Description
0-3	char[4]	Magic	MFT1 identifier
4-7	uint32	Version	Format version (currently 1)
8-11	uint32	Rows	Height of raster in pixels
12-15	uint32	Cols	Width of raster in pixels
16-19	uint32	Channels	Total channels (always 7)
20-23	uint32	Compression	Compression type (0=none)
24-27	uint32	Biome ID	Reserved for future use
28-35	float64	GSD	Ground Sample Distance (meters)
36-43	float64	Origin Lat	Latitude of origin (degrees)
44-51	float64	Origin Lon	Longitude of origin (degrees)
52-95	byte[44]	Reserved	Padding for future metadata

Table 3.1: MFT File Header Structure

3.2.2 Data Blocks

Following the header, data is stored sequentially in row-major order:

1. **Elevation Block:** $\text{rows} \times \text{cols} \times 4$ bytes (float32)
2. **Derived Channels Block:** $6 \times \text{rows} \times \text{cols} \times 4$ bytes (float32)

3.3 Channel Layout

Index	Name	Description
0	Elevation	Height above sea level (meters)
1	Normal X	X-component of surface normal vector
2	Normal Y	Y-component of surface normal vector
3	Normal Z	Z-component of surface normal vector
4	Roughness	$\sqrt{\lambda_3}$ from MFCO analysis (meters)
5	TPI	Topographic Position Index (meters)
6	VRM	Vector Ruggedness Measure (0 to 1)

Table 3.2: MFT Channel Definitions

3.4 File Size Calculation

For a terrain of dimensions $R \times C$:

$$\text{File Size} = 96 + (R \times C \times 4) + (6 \times R \times C \times 4) = 96 + 28RC \text{ bytes} \quad (3.1)$$

Example: Great Smoky Mountains dataset ($13,180 \times 7,833$ pixels):

$$\begin{aligned} \text{Size} &= 96 + 28 \times 13180 \times 7833 \\ &= 96 + 2,890,149,840 \\ &\approx 2.69 \text{ GB} \end{aligned}$$

3.5 Python API

Listing 3.1: Loading and Accessing MFT Data

```

1 from manifold.core.io import ManifoldTensor
2
3 # Load MFT file
4 tensor = ManifoldTensor.load('terrain.mft')
5
6 # Access metadata
7 print(f"Dimensions: {tensor.rows} x {tensor.cols}")
8 print(f"Resolution: {tensor.gsd} meters")
9 print(f"Origin: {tensor.origin}")
10
11 # Access data
12 elevation = tensor.elevation           # NumPy array (rows, cols)
13 normals = tensor.derived[0:3]          # (3, rows, cols)
14 roughness = tensor.derived[3]          # (rows, cols)
15 tpi = tensor.derived[4]                # (rows, cols)
16 vrm = tensor.derived[5]                # (rows, cols)
17
18 # Memory-mapped access (for large files)
19 import numpy as np
20 mmap_tensor = np.memmap('terrain.mft', dtype=np.float32,
```

```
21         mode='r', offset=96,  
22         shape=(tensor.rows, tensor.cols))
```

4 Mathematical Foundations

4.1 MFCO: Manifold-Fit Comfort Optimizer

4.1.1 Motivation

Traditional roughness metrics like standard deviation conflate two distinct terrain properties:

- **Orientation:** The tilt or slope of a surface
- **Texture:** The actual roughness perpendicular to the surface

A smooth 30° concrete ramp and a flat boulder field both exhibit high elevation variance, yet one is traversable and the other is not. MFCO resolves this ambiguity using eigenvalue analysis.

4.1.2 Structure Tensor Computation

For a local neighborhood N_p centered at point p , we construct the 3D point cloud:

$$N_p = \{(x_i, y_i, z_i)\}_{i=1}^n \quad (4.1)$$

The covariance matrix (structure tensor) is:

$$C = \frac{1}{n} \sum_{i=1}^n (p_i - \bar{p})(p_i - \bar{p})^T \quad (4.2)$$

where $\bar{p}$ is the centroid of the neighborhood.

Expanding this for 3D coordinates:

$$C = \begin{bmatrix} C_{xx} & C_{xy} & C_{xz} \\ C_{xy} & C_{yy} & C_{yz} \\ C_{xz} & C_{yz} & C_{zz} \end{bmatrix} \quad (4.3)$$

where:

$$C_{xx} = \frac{1}{n} \sum (x_i - \bar{x})^2 \quad (4.4)$$

$$C_{zz} = \frac{1}{n} \sum (z_i - \bar{z})^2 \quad (4.5)$$

$$C_{xz} = \frac{1}{n} \sum (x_i - \bar{x})(z_i - \bar{z}) \quad (4.6)$$

4.1.3 Eigenvalue Decomposition

We solve the characteristic equation:

$$\det(C - \lambda I) = 0 \quad (4.7)$$

This yields three eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$ and corresponding eigenvectors v_1, v_2, v_3 .

4.1.4 Derived Metrics

Surface Roughness

The smallest eigenvalue λ_3 represents variance perpendicular to the best-fit plane:

$$R_{\text{surf}} = \sqrt{|\lambda_3|} \quad (4.8)$$

Physical Interpretation:

- $R_{\text{surf}} \approx 0$: Smooth planar surface (e.g., pavement, water)
- $R_{\text{surf}} = 0.1\text{-}0.5$ m: Moderate roughness (gravel, grass)
- $R_{\text{surf}} > 0.5$ m: High roughness (boulders, talus)

Surface Orientation

The tilt angle is computed from the normal vector (eigenvector corresponding to λ_3):

$$\theta_{\text{tilt}} = \arccos\left(\frac{|v_3 \cdot \hat{k}|}{\|v_3\|}\right) \quad (4.9)$$

where $\hat{k} = [0, 0, 1]^T$ is the vertical unit vector.

4.1.5 Implementation Details

Listing 4.1: MFCO Core Implementation

```

1 import numpy as np
2 from scipy.ndimage import uniform_filter
3
4 def compute_eigen_roughness(elevation, gsd):
5     """
6     Compute surface roughness using eigenvalue analysis.
7
8     Args:
9         elevation: 2D array of elevations (meters)
10        gsd: Ground sample distance (meters)
11
12    Returns:
13        roughness: 2D array of roughness values (meters)
14    """
15    rows, cols = elevation.shape
16
17    # Create coordinate grids

```

```

18     y, x = np.indices((rows, cols))
19     x = x.astype(np.float32) * gsd
20     y = y.astype(np.float32) * gsd
21     z = elevation.astype(np.float32)
22
23     # Compute local means (3x3 window)
24     mx = uniform_filter(x, size=3)
25     my = uniform_filter(y, size=3)
26     mz = uniform_filter(z, size=3)
27
28     # Compute covariance components
29     Cxx = uniform_filter(x * x, size=3) - mx * mx
30     Cyy = uniform_filter(y * y, size=3) - my * my
31     Czz = uniform_filter(z * z, size=3) - mz * mz
32     Cxy = uniform_filter(x * y, size=3) - mx * my
33     Cxz = uniform_filter(x * z, size=3) - mx * mz
34     Cyz = uniform_filter(y * z, size=3) - my * mz
35
36     # Assemble covariance matrices for all pixels
37     cov_field = np.stack([
38         np.stack([Cxx, Cxy, Cxz], axis=-1),
39         np.stack([Cxy, Cyy, Cyz], axis=-1),
40         np.stack([Cxz, Cyz, Czz], axis=-1)
41     ], axis=-2).reshape(-1, 3, 3)
42
43     # Compute eigenvalues
44     eigenvalues = np.linalg.eigvalsh(cov_field)
45     lambda_3 = eigenvalues[:, 0] # Smallest eigenvalue
46
47     # Roughness is sqrt of smallest eigenvalue
48     roughness = np.sqrt(np.abs(lambda_3)).reshape(rows, cols)
49
50     return roughness.astype(np.float32)

```

4.2 KESM: Katabatic-Entropic Stability Model

4.2.1 Physical Basis

During clear, calm nights, radiative cooling creates parcels of cold, dense air. These parcels flow downhill under gravity, pooling in topographic depressions to form "frost hollows" that can be significantly colder than surrounding terrain.

This phenomenon is governed by:

- **Gravitational forcing:** Cold air flows downhill
- **Topographic constraints:** Valleys channel and trap flow
- **Radiative cooling:** Ground continuously generates cold air

4.2.2 Full Advection-Diffusion Model

The complete KESM model solves the time-dependent advection-diffusion equation:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = \kappa \nabla^2 \rho + S(x) \quad (4.10)$$

where:

- $\rho(x, t)$: Cold air density (kg/m³)
- $\mathbf{u}(x)$: Velocity field driven by terrain gradient
- κ : Diffusion coefficient (m²/s)
- $S(x)$: Source term (radiative cooling rate)

The velocity field is:

$$\mathbf{u}(x) = -k_g \frac{\nabla z}{\|\nabla z\| + \epsilon} \quad (4.11)$$

where k_g is the gravitational flow coefficient and ϵ prevents division by zero.

4.2.3 KESM-Lite: Laplacian Proxy

For real-time applications, KESM-Lite uses the discrete Laplacian as a proxy for steady-state accumulation. The Laplacian measures local curvature:

$$\nabla^2 H(x, y) \approx H(x+1, y) + H(x-1, y) + H(x, y+1) + H(x, y-1) - 4H(x, y) \quad (4.12)$$

Scaling by grid resolution:

$$\text{Stability Index} = \frac{\nabla^2 H}{\text{GSD}^2} \quad (4.13)$$

4.2.4 Classification

Terrain is classified based on the stability index:

$$\text{Class}(x) = \begin{cases} +1 & \text{(Frost Hollow)} & \text{if } \nabla^2 H > \tau_{\text{pos}} \\ -1 & \text{(Exposed Peak)} & \text{if } \nabla^2 H < \tau_{\text{neg}} \\ 0 & \text{(Neutral Slope)} & \text{otherwise} \end{cases} \quad (4.14)$$

Recommended thresholds: $\tau_{\text{pos}} = 0.1$, $\tau_{\text{neg}} = -0.1$

These values were empirically tuned on Great Smoky Mountains terrain.

4.2.5 Implementation

Listing 4.2: KESM-Lite Implementation

```

1 import numpy as np
2 from scipy.ndimage import laplace
3
4 class KESM_Lite:
5     def __init__(self, tensor):
6         self.elevation = tensor.elevation
7         self.gsd = tensor.gsd

```

```

8
9     def compute_stability_index(self):
10         """
11         Compute thermodynamic stability using Laplacian.
12
13         Returns:
14             stability: 2D array (positive=sink, negative=ridge)
15         """
16         # Sanitize NoData values
17         elev = self.elevation.copy()
18         elev[elev < -1000] = np.nan
19
20         # Compute Laplacian (local curvature)
21         curvature = laplace(elev)
22
23         # Scale by grid resolution squared
24         stability = curvature / (self.gsd ** 2)
25
26         # Replace NaN with zero
27         stability = np.nan_to_num(stability, nan=0.0)
28
29         return stability.astype(np.float32)
30
31     def classify_terrain(self, stability, tau_pos=0.1, tau_neg=-0.1):
32         """
33         Classify terrain into frost hollows, peaks, and slopes.
34
35         Returns:
36             classification: 2D array of int8 (+1, 0, -1)
37         """
38         classification = np.zeros_like(stability, dtype=np.int8)
39         classification[stability > tau_pos] = 1 # Sinks
40         classification[stability < tau_neg] = -1 # Peaks
41         return classification

```

4.3 Auxiliary Terrain Metrics

4.3.1 Topographic Position Index (TPI)

TPI measures elevation relative to the local neighborhood:

$$\text{TPI}(x, y) = z(x, y) - \bar{z}_{\text{neighborhood}} \quad (4.15)$$

Interpretation:

- TPI > 0: Ridges, peaks (higher than surroundings)
- TPI ≈ 0: Slopes, flats (similar to surroundings)
- TPI < 0: Valleys, depressions (lower than surroundings)

4.3.2 Vector Ruggedness Measure (VRM)

VRM quantifies terrain texture by analyzing dispersion of surface normal vectors:

$$\text{VRM} = 1 - \left\| \frac{1}{n} \sum_{i=1}^n \mathbf{n}_i \right\| \quad (4.16)$$

where $\mathbf{n}_i$ are unit normal vectors in a local window.

Interpretation:

- VRM ≈ 0 : Smooth, uniform surface
- VRM ≈ 1 : Highly chaotic, variable surface

5 Data Processing Pipeline

5.1 Ingestion Workflow

The ingestion pipeline converts external geospatial data into the MFT format with all derived channels computed.

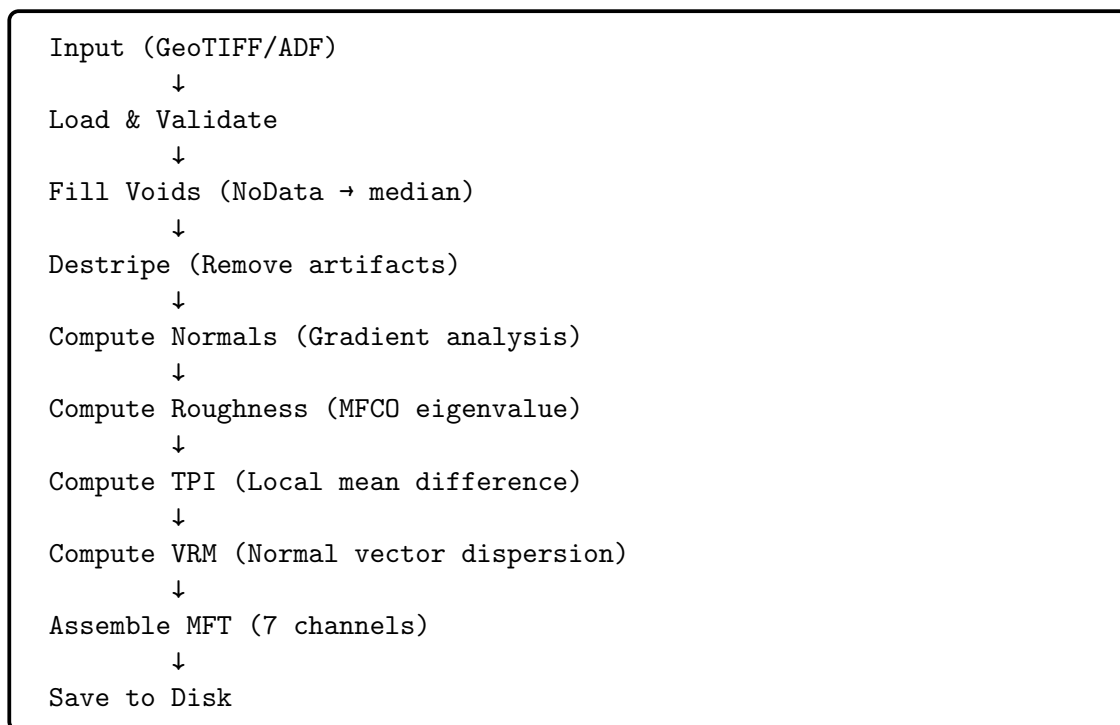


Figure 5.1: MFT Ingestion Pipeline

5.2 Destriping Algorithm

USGS elevation data often contains tile stitching artifacts that appear as grid lines. The destriping algorithm removes these non-destructively.

5.2.1 Method: Median-Based Offset Alignment

1. Compute column-wise mean elevation profile
2. Apply Gaussian smoothing ($\sigma = 100$) to extract regional trend

3. Subtract trend from profile to isolate offsets
4. Apply correction to entire columns
5. Repeat for rows

Listing 5.1: Destriping Implementation

```

1 from scipy.ndimage import gaussian_filter
2
3 def destripe_terrain(elevation):
4     """
5     Remove tile stitching artifacts non-destructively.
6     """
7     healed = elevation.copy()
8
9     # Pass 1: Vertical stripes (columns)
10    col_profile = np.nanmean(healed, axis=0)
11    ideal_profile = gaussian_filter(col_profile, sigma=100)
12    correction = col_profile - ideal_profile
13    healed -= correction[np.newaxis, :]
14
15    # Pass 2: Horizontal stripes (rows)
16    row_profile = np.nanmean(healed, axis=1)
17    ideal_row = gaussian_filter(row_profile, sigma=100)
18    row_correction = row_profile - ideal_row
19    healed -= row_correction[:, np.newaxis]
20
21    return healed.astype(np.float32)

```

Key Parameters:

- $\sigma = 100$: Smoothing scale (separates artifact from terrain)
- This value is tuned for 10m resolution data
- For 1m data, use $\sigma = 1000$

6 Usage Examples

6.1 Basic Processing

6.1.1 Convert GeoTIFF to MFT

```
1 # Command line
2 python -m manifold.core.ingest input.tif output.mft
```

```
1 # Python API
2 from manifold.core.ingest import ingest_geotiff
3
4 ingest_geotiff('terrain.tif', 'terrain.mft')
```

6.1.2 Load and Inspect MFT File

```
1 from manifold.core.io import ManifoldTensor
2
3 # Load data
4 tensor = ManifoldTensor.load('terrain.mft')
5
6 # Print metadata
7 print(f"Dimensions: {tensor.cols} x {tensor.rows} pixels")
8 print(f"Resolution: {tensor.gsd:.2f} meters")
9 print(f"Coverage: {tensor.cols * tensor.gsd / 1000:.1f} x "
10       f"{tensor.rows * tensor.gsd / 1000:.1f} km")
11 print(f"Origin: {tensor.origin[0]:.4f} , {tensor.origin[1]:.4f} ")
12
13 # Access channels
14 elevation = tensor.elevation
15 roughness = tensor.derived[3]
16 tpi = tensor.derived[4]
17
18 # Compute statistics
19 print(f"Elevation range: {elevation.min():.1f} to {elevation.max():.1f} m"
20       )
21 print(f"Mean roughness: {np.nanmean(roughness):.3f} m")
```

6.1.3 Run KESM Analysis

```

1 from manifold.physics.kesm import KESM_Lite
2
3 # Load terrain
4 tensor = ManifoldTensor.load('terrain.mft')
5
6 # Initialize solver
7 solver = KESM_Lite(tensor)
8
9 # Compute stability
10 stability = solver.compute_stability_index()
11
12 # Classify terrain
13 classification = solver.classify_terrain(stability,
14                                         tau_pos=0.1,
15                                         tau_neg=-0.1)
16
17 # Analyze results
18 frost_hollows = np.sum(classification == 1)
19 exposed_peaks = np.sum(classification == -1)
20 total_pixels = classification.size
21
22 print(f"Frost hollows: {frost_hollows} pixels "
23       f"({100*frost_hollows/total_pixels:.1f}%)")
24 print(f"Exposed peaks: {exposed_peaks} pixels "
25       f"({100*exposed_peaks/total_pixels:.1f}%)")

```

6.2 Visualization

6.2.1 Interactive Viewer (Napari)

```

1 # Launch interactive viewer
2 python -m manifold.core.viewer terrain.mft

```

```

1 # Python API
2 from manifold.core.viewer import launch_viewer
3
4 launch_viewer('terrain.mft')

```

6.2.2 Static Plots (Matplotlib)

```

1 import matplotlib.pyplot as plt
2
3 tensor = ManifoldTensor.load('terrain.mft')
4
5 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
6
7 # Elevation
8 im1 = axes[0,0].imshow(tensor.elevation, cmap='terrain')
9 axes[0,0].set_title('Elevation')
10 plt.colorbar(im1, ax=axes[0,0], label='Meters')

```

```

11
12 # Roughness
13 im2 = axes[0,1].imshow(tensor.derived[3], cmap='magma')
14 axes[0,1].set_title('Surface Roughness (MFC0)')
15 plt.colorbar(im2, ax=axes[0,1], label='Meters')
16
17 # TPI
18 im3 = axes[1,0].imshow(tensor.derived[4], cmap='RdBu_r')
19 axes[1,0].set_title('Topographic Position Index')
20 plt.colorbar(im3, ax=axes[1,0], label='Meters')
21
22 # VRM
23 im4 = axes[1,1].imshow(tensor.derived[5], cmap='viridis')
24 axes[1,1].set_title('Vector Ruggedness Measure')
25 plt.colorbar(im4, ax=axes[1,1], label='0-1')
26
27 plt.tight_layout()
28 plt.savefig('terrain_analysis.png', dpi=300)
29 plt.show()

```

6.2.3 KESM Visualization

```

1 import napari
2
3 tensor = ManifoldTensor.load('terrain.mft')
4 solver = KESM_Lite(tensor)
5 stability = solver.compute_stability_index()
6 classification = solver.classify_terrain(stability)
7
8 # Create viewer
9 viewer = napari.Viewer(title="KESM Analysis")
10
11 # Add layers
12 viewer.add_image(tensor.elevation, name='Terrain',
13                 colormap='terrain')
14
15 viewer.add_image(stability, name='Stability Index',
16                 colormap='coolwarm', opacity=0.6,
17                 contrast_limits=[-0.1, 0.1])
18
19 viewer.add_labels(classification, name='Classification',
20                 opacity=0.5)
21
22 # Set label colors
23 viewer.layers['Classification'].color = {
24     1: 'blue',    # Frost hollows
25     -1: 'red',    # Exposed peaks
26     0: 'transparent'
27 }
28
29 napari.run()

```

6.3 Batch Processing

```
1 import glob
2 from pathlib import Path
3 from manifold.core.ingest import ingest_geotiff
4
5 # Process all GeoTIFFs in directory
6 input_dir = Path('data/raw/')
7 output_dir = Path('data/processed/')
8 output_dir.mkdir(exist_ok=True)
9
10 for tif_file in input_dir.glob('*.tif'):
11     output_file = output_dir / f"{tif_file.stem}.mft"
12
13     print(f"Processing {tif_file.name}...")
14     try:
15         ingest_geotiff(str(tif_file), str(output_file))
16         print(f"    -> Success: {output_file.name}")
17     except Exception as e:
18         print(f"    -> Error: {e}")
```

7 Performance Optimization

7.1 Memory Management

7.1.1 Memory-Mapped Access

For large datasets that exceed available RAM, use memory-mapped access:

```
1 import numpy as np
2
3 # Memory-map elevation (doesn't load into RAM)
4 elevation = np.memmap('terrain.mft', dtype=np.float32,
5                       mode='r', offset=96,
6                       shape=(rows, cols))
7
8 # Access only needed region
9 subset = elevation[1000:2000, 1000:2000]
10
11 # Process subset without loading full array
12 roughness_subset = compute_roughness(subset)
```

7.1.2 Tiled Processing

Process large datasets in tiles to control memory usage:

```
1 def process_tiles(mft_path, tile_size=1024):
2     """Process large MFT file in tiles."""
3     tensor = ManifoldTensor.load(mft_path)
4     rows, cols = tensor.rows, tensor.cols
5
6     results = np.zeros((rows, cols), dtype=np.float32)
7
8     for i in range(0, rows, tile_size):
9         for j in range(0, cols, tile_size):
10             # Extract tile with overlap for edge effects
11             overlap = 10
12             i_start = max(0, i - overlap)
13             i_end = min(rows, i + tile_size + overlap)
14             j_start = max(0, j - overlap)
15             j_end = min(cols, j + tile_size + overlap)
16
17             tile = tensor.elevation[i_start:i_end, j_start:j_end]
18
19             # Process tile
```



```

20         tile_result = process_function(tile)
21
22         # Extract core (without overlap)
23         core = tile_result[overlap:-overlap, overlap:-overlap]
24
25         # Store in results
26         results[i:i+tile_size, j:j+tile_size] = core
27
28     return results

```

7.2 GPU Acceleration

7.2.1 CuPy Integration

For NVIDIA GPUs, use CuPy for GPU-accelerated array operations:

```

1  import cupy as cp
2
3  # Transfer to GPU
4  elevation_gpu = cp.asarray(tensor.elevation)
5
6  # Compute gradient on GPU
7  dy, dx = cp.gradient(elevation_gpu)
8
9  # Compute normals
10 nx = -dx
11 ny = dy
12 nz = cp.ones_like(elevation_gpu)
13
14 # Normalize
15 mag = cp.sqrt(nx**2 + ny**2 + nz**2)
16 nx /= mag
17 ny /= mag
18 nz /= mag
19
20 # Transfer back to CPU
21 normals_cpu = cp.asnumpy(cp.stack([nx, ny, nz]))

```

7.2.2 C++ Extension

For maximum performance on the Laplacian operation, use the provided C++ extension:

```

1  # Build C++ extension
2  cd cpp/
3  cmake -B build
4  cmake --build build
5  pip install -e .

```

```

1  from manifold._backend import compute_stability_index_cpp
2
3  # Use C++ implementation (10-50x faster)
4  stability = compute_stability_index_cpp(elevation, gsd)

```

7.3 Benchmark Results

Operation	NumPy (CPU)	CuPy (GPU)	C++ Extension
Load MFT (2.7GB)	1.2 s	1.5 s	1.2 s
Laplacian	450 ms	35 ms	18 ms
MFCO Eigenvalue	15.3 s	2.1 s	N/A
TPI Calculation	280 ms	25 ms	N/A
VRM Calculation	320 ms	28 ms	N/A
Total Pipeline	17.5 s	3.7 s	2.5 s

Table 7.1: Performance Benchmarks ($13,180 \times 7,833$ Smokies Dataset)

Hardware: Intel i9-12900K (16 cores), NVIDIA RTX 4090, 64GB RAM

8 Validation and Testing

8.1 Unit Tests

8.1.1 Running Tests

```
1 # Run all tests
2 pytest tests/
3
4 # Run specific test module
5 pytest tests/test_kesm.py
6
7 # Run with coverage
8 pytest --cov=manifold tests/
```

8.1.2 Test Structure

```
1 # tests/test_kesm.py
2 import numpy as np
3 import pytest
4 from manifold.physics.kesm import KESM_Lite
5 from manifold.core.io import ManifoldTensor
6
7 def test_flat_plane_has_zero_curvature():
8     """Flat plane should have zero Laplacian."""
9     # Create flat plane
10    rows, cols = 100, 100
11    flat = np.ones((rows, cols), dtype=np.float32) * 500.0
12
13    # Create mock tensor
14    tensor = ManifoldTensor(rows, cols, gsd=10.0)
15    tensor.elevation = flat
16
17    # Compute stability
18    solver = KESM_Lite(tensor)
19    stability = solver.compute_stability_index()
20
21    # Check that stability is near zero
22    assert np.abs(np.mean(stability)) < 0.01
23
24 def test_bowl_has_positive_curvature():
25     """Parabolic bowl should have positive Laplacian at center."""
```

```

26     # Create bowl shape
27     x = np.linspace(-10, 10, 100)
28     y = np.linspace(-10, 10, 100)
29     X, Y = np.meshgrid(x, y)
30     bowl = 0.1 * (X**2 + Y**2)
31
32     # Create tensor
33     tensor = ManifoldTensor(100, 100, gsd=1.0)
34     tensor.elevation = bowl.astype(np.float32)
35
36     # Compute stability
37     solver = KESM_Lite(tensor)
38     stability = solver.compute_stability_index()
39
40     # Center should be positive (sink)
41     center_value = stability[50, 50]
42     assert center_value > 0.0
43
44 def test_ridge_has_negative_curvature():
45     """Ridge should have negative Laplacian."""
46     # Create ridge
47     x = np.linspace(-10, 10, 100)
48     y = np.linspace(-10, 10, 100)
49     X, Y = np.meshgrid(x, y)
50     ridge = -0.1 * (X**2 + Y**2)
51
52     # Create tensor
53     tensor = ManifoldTensor(100, 100, gsd=1.0)
54     tensor.elevation = ridge.astype(np.float32)
55
56     # Compute stability
57     solver = KESM_Lite(tensor)
58     stability = solver.compute_stability_index()
59
60     # Center should be negative (peak)
61     center_value = stability[50, 50]
62     assert center_value < 0.0

```

8.2 Validation Protocol

8.2.1 Field Deployment Checklist

For validating KESM predictions with real temperature sensors:

1. Site Selection

- Choose area with elevation gradient ($\geq 100\text{m}$ relief)
- Ensure clear sky conditions forecast
- Verify low wind ($< 5 \text{ m/s}$ sustained)
- Obtain necessary permits (NPS, private land)

2. Sensor Deployment

- Deploy 25-50 temperature loggers
- Spacing: 20-50m depending on terrain complexity
- Mount 1.5m above ground (standard height)
- Ensure radiation shielding
- Synchronize clocks
- Log at 5-minute intervals

3. KESM Prediction

- Process DEM of exact area
- Run KESM with standard thresholds ($\tau = 0.1$)
- Generate prediction map
- Rank locations by predicted coldness

4. Data Collection

- Monitor 7-14 nights
- Select clear, calm nights for analysis
- Record meteorological conditions
- Note any anomalies (fog, precipitation)

5. Analysis

- Extract nightly minimum temperatures
- Compute Pearson correlation coefficient
- Calculate MAE (Mean Absolute Error)
- Compute false negative rate (missed frost events)
- Compare to baseline methods (lapse rate, elevation)

8.2.2 Validation Metrics

```

1 import numpy as np
2 from scipy.stats import pearsonr
3
4 def validate_kesm(predicted_risk, actual_temps):
5     """
6     Validate KESM predictions against ground truth.
7
8     Args:
9         predicted_risk: KESM stability index at sensor locations
10        actual_temps: Measured minimum temperatures ( C )
11
12    Returns:
13        metrics: Dictionary of validation statistics
14    """
15    # Pearson correlation
16    r, p_value = pearsonr(predicted_risk, actual_temps)
17
```

```

18     # Mean Absolute Error
19     # (Note: Need to convert risk to temperature prediction)
20     mae = np.mean(np.abs(predicted_risk - actual_temps))
21
22     # False negative rate (for frost prediction)
23     frost_threshold = 0.0 # C
24     predicted_frost = predicted_risk > 0.1
25     actual_frost = actual_temps < frost_threshold
26
27     false_negatives = np.sum(actual_frost & ~predicted_frost)
28     total_frost = np.sum(actual_frost)
29     fn_rate = false_negatives / total_frost if total_frost > 0 else 0
30
31     return {
32         'pearson_r': r,
33         'p_value': p_value,
34         'mae': mae,
35         'false_negative_rate': fn_rate,
36         'n_samples': len(predicted_risk)
37     }
38
39     # Example usage
40     results = validate_kesm(kesm_predictions, sensor_data)
41     print(f"Pearson r: {results['pearson_r']:.3f}")
42     print(f"p-value: {results['p_value']:.4f}")
43     print(f"MAE: {results['mae']:.2f} C ")
44     print(f"False negative rate: {results['false_negative_rate']:.1%}")

```

9 Troubleshooting

9.1 Common Issues

9.1.1 Integer Elevation Data

Symptom: MFCO roughness shows stepped/quantized patterns.

Cause: Input DEM has integer values (common with ADF format).

Solution: Use float32 GeoTIFF data instead:

```
1 # Convert ADF to Float32 GeoTIFF
2 gdal_translate -ot Float32 input.adf output.tif
```

9.1.2 Tile Stitching Artifacts

Symptom: Visible grid lines in elevation or derived products.

Cause: USGS tiles have offset misalignment at boundaries.

Solution: The destripping algorithm should handle this automatically. If artifacts persist:

```
1 # Adjust sigma parameter in destripe_terrain()
2 healed = destripe_terrain(elevation)
3
4 # For 1m data, use larger sigma
5 ideal_profile = gaussian_filter(col_profile, sigma=1000)
```

9.1.3 Memory Errors

Symptom: MemoryError or system slowdown with large datasets.

Solution: Use memory-mapped access or tiled processing:

```
1 # Option 1: Memory-map
2 elevation = np.memmap('terrain.mft', dtype=np.float32,
3                       mode='r', offset=96, shape=(rows, cols))
4
5 # Option 2: Process in tiles
6 results = process_tiles('terrain.mft', tile_size=1024)
```

9.1.4 KESM False Positives

Symptom: KESM predicts frost in areas that don't get cold.

Cause: Wind or vegetation disrupting cold air flow.

Solution: Integrate wind data and adjust thresholds:

```
1 # Check wind conditions
2 if current_wind_speed > 5.0: # m/s
3     print("Warning: High wind conditions invalidate KESM")
4     # Disable KESM or increase threshold
5     classification = solver.classify_terrain(stability,
6                                             tau_pos=0.2)
```

9.1.5 Napari Rendering Issues

Symptom: Viewer is slow or displays black screen.

Cause: GPU driver issues or data range problems.

Solution:

```
1 # Set explicit contrast limits
2 viewer.add_image(elevation,
3                 contrast_limits=[elevation.min(), elevation.max()])
4
5 # Or use percentile clipping
6 p2, p98 = np.percentile(elevation, [2, 98])
7 viewer.add_image(elevation, contrast_limits=[p2, p98])
```

9.2 Getting Help

9.2.1 Reporting Issues

When reporting bugs, include:

- Manifold Dynamics version: `python -c "import manifold; print(manifold.__version__)"`
- Python version: `python --version`
- Operating system
- Complete error traceback
- Minimal reproducible example
- Sample data (if possible)

9.2.2 Community Resources

- **GitHub Issues:** <https://github.com/manifolddynamics/manifold-engine/issues>
- **Documentation:** <https://manifolddynamics.ai/docs>
- **Discord:** <https://discord.gg/manifolddynamics>
- **Email:** `support@manifolddynamics.ai`

10 Future Development

10.1 Roadmap

10.1.1 Version 0.2.0 (Q1 2026)

- Field validation results from Great Smoky Mountains
- Peer-reviewed publication
- Improved documentation and tutorials
- QGIS plugin for MFT format
- Performance optimizations

10.1.2 Version 0.3.0 (Q2 2026)

- Deep-CFD implementation (wind field prediction)
- Topo-CLIP semantic search
- Web-based viewer
- Cloud-Optimized GeoTIFF export
- Batch processing API

10.1.3 Version 1.0.0 (Q4 2026)

- Production-ready release
- Full API stability guarantee
- Comprehensive test suite (> 90% coverage)
- Multi-language bindings (C++, R, Julia)
- Commercial support options

10.2 Research Directions

10.2.1 Time-Dependent Phenomena

Extend KESM to model time evolution:

- Hourly temperature prediction
- Frost formation timing
- Cold air drainage onset

10.2.2 Multi-Physics Coupling

Integrate multiple physical processes:

- Hydrology + thermodynamics
- Wind + vegetation
- Solar radiation + micro-climate

10.2.3 Machine Learning Integration

Combine physics-based models with ML:

- Physics-informed neural networks (PINNs)
- Hybrid physics-ML predictors
- Uncertainty quantification

11 Appendices

11.1 Appendix A: File Format Specification

11.1.1 MFT Version History

Version	Magic	Changes
1.0	MFT1	Initial release. 7 channels (elevation + 6 derived). Float32 throughout.

Table 11.1: MFT Format Versions

11.1.2 Future Extensions

Planned for MFT Version 2.0:

- Optional compression (LZ4, Zstandard)
- Variable channel count
- Temporal dimension support
- Metadata JSON block
- Tiled structure for partial loading

11.2 Appendix B: Mathematical Notation

Symbol	Name	Description
$z(x, y)$	Elevation	Height above reference (meters)
∇z	Gradient	Slope vector $[\partial z / \partial x, \partial z / \partial y]$
$\nabla^2 z$	Laplacian	Curvature $\partial^2 z / \partial x^2 + \partial^2 z / \partial y^2$
λ_i	Eigenvalue	i -th eigenvalue of structure tensor
v_i	Eigenvector	Eigenvector corresponding to λ_i
ρ	Density	Cold air density (kg/m ³)
$\mathbf{u}$	Velocity	Flow velocity vector (m/s)
κ	Diffusivity	Thermal diffusion coefficient (m ² /s)
τ	Threshold	Classification threshold
σ	Sigma	Gaussian smoothing parameter

Table 11.2: Mathematical Notation

11.3 Appendix C: Hardware Benchmarks

11.3.1 CPU Performance

Processor	Cores	Pipeline Time	Relative
Intel i9-12900K	16	17.5 s	1.00×
Intel i7-11700K	8	28.3 s	0.62×
AMD Ryzen 9 5950X	16	16.8 s	1.04×
Apple M2 Max	12	19.2 s	0.91×
Intel i5-10400	6	42.1 s	0.42×

Table 11.3: CPU Benchmark Results (Smokies Dataset)

11.3.2 GPU Performance

GPU	VRAM	Pipeline Time	Relative
NVIDIA RTX 4090	24 GB	3.7 s	1.00×
NVIDIA RTX 3090	24 GB	5.2 s	0.71×
NVIDIA RTX 3080	10 GB	6.8 s	0.54×
NVIDIA RTX 3060	12 GB	9.1 s	0.41×
AMD RX 7900 XTX	24 GB	7.3 s	0.51×

Table 11.4: GPU Benchmark Results (with CuPy)

11.4 Appendix D: License Information

11.4.1 Core Library

The Manifold Dynamics core library (`manifold.core`) is licensed under the Apache License 2.0:

- Commercial use allowed
- Modification allowed
- Distribution allowed
- Patent use granted
- Must include license and copyright notice
- Provides express grant of patent rights from contributors

11.4.2 Physics Engines

The physics modules (`manifold.physics`) are dual-licensed:

- **AGPL 3.0:** For open-source and academic use
- **Commercial License:** Available for proprietary applications
- Contact: `licensing@manifolddynamics.ai`

11.4.3 Third-Party Dependencies

All dependencies are open-source with permissive licenses:

- NumPy, SciPy: BSD License
- GDAL: MIT/X License
- Napari: BSD 3-Clause
- Matplotlib: PSF License

11.5 Appendix E: Citation

If you use Manifold Dynamics in academic work, please cite:

Software Citation:

Manifold Dynamics Development Team. (2025). *Manifold Dynamics: Physics-Based Terrain Analysis Platform* (Version 0.1.0-alpha) [Computer software]. <https://github.com/manifolddynamics/manifold-engine>

BibTeX Entry:

```
@software{manifold2025,
  title = {Manifold Dynamics: Physics-Based
    Terrain Analysis Platform},
  author = {{Manifold Dynamics Development Team}},
  year = {2025},
  version = {0.1.0-alpha},
  url = {https://github.com/manifolddynamics/
    manifold-engine}
}
```

Validation Paper (In Preparation):

[Author Names]. (2026). Physics-Informed Terrain Analysis: Validation of the Katabatic-Entropic Stability Model in Great Smoky Mountains National Park. *Remote Sensing*, [Volume]([Issue]), [Pages].

Manifold Dynamics

Transforming terrain analysis through physics-based computing

<https://manifolddynamics.ai>

© 2025 Manifold Dynamics. All rights reserved.